

Agents as a New system Workload

Ruijie Gao, Xinyi Xu, Chenglin Li

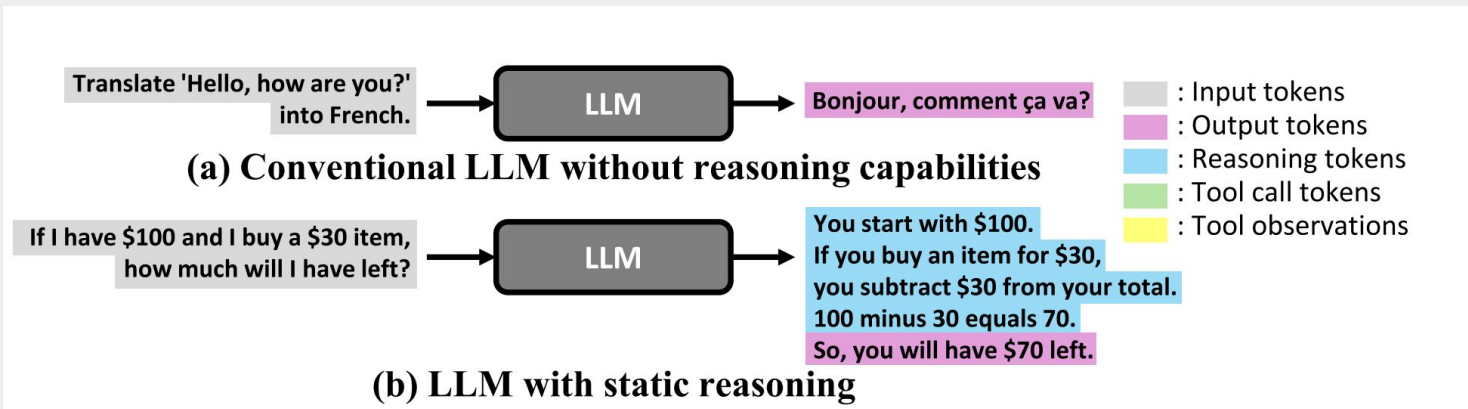
The Cost of Dynamic Reasoning: Demystifying AI Agents and Test-Time Scaling from an AI Infrastructure Perspective

Giin Kim, Byeongjun Shin, Jinha Chung, Minsoo Rhu

Background and Motivation

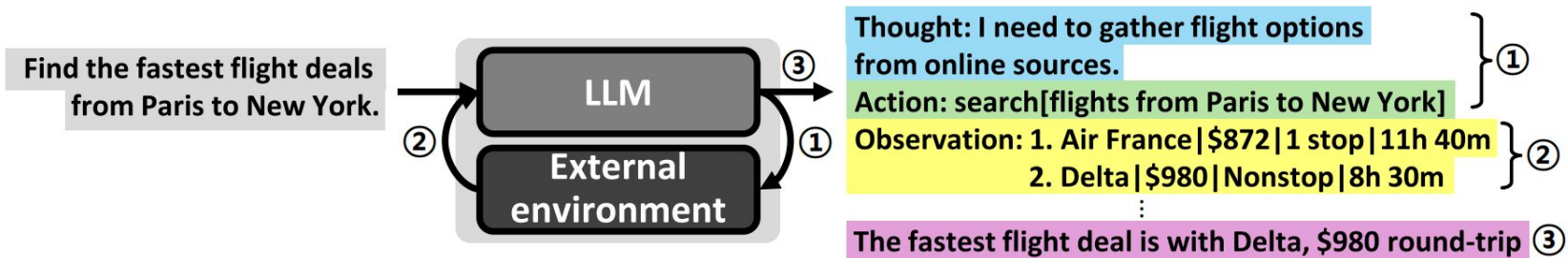
From Test-Time Scaling to Dynamic Reasoning

- Test-time scaling improves model performance by spending more compute during inference.
- Examples: Chain-of-Thought (CoT) and Tree-of-Thought (ToT).
- AI agents extend this idea with iterative reasoning and external actions.



AI Agents Turn Reasoning into a Dynamic Workflow

- Agents iteratively:
 - Reason → Act / Call Tool → Observe → Adapt
- Intermediate outcomes determine what happens next.
- The execution path evolves dynamically at runtime.



(c) AI agents with dynamic reasoning

Dynamic Reasoning Changes the Serving Workload

Dynamic control flow
Execution path changes
at runtime

Repeated LLM calls
More inference within one
request

Tool interactions
External actions join the
critical path

These behaviors create variable compute, memory pressure, and latency → motivating a system-level characterization.

Methodology

Methodology

Five representative agents

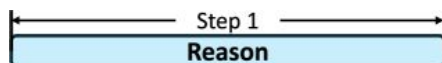
Agent	Reasoning	Tool use	Reflection	Tree search	Planning
CoT	✓	—	—	—	—
ReAct	✓	✓	—	—	—
Reflexion	✓	✓	✓	—	—
LATS	✓	✓	✓	✓	—
LLMCompiler	✓	✓	✓	—	✓

Tasks: HotpotQA, WebShop, MATH, HumanEval · **Baseline:** ShareGPT chatbot

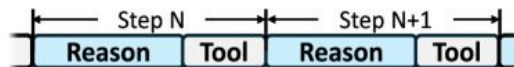
Methodology

How each agent runs

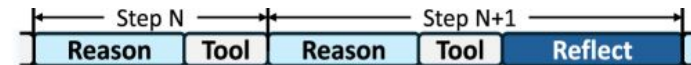
CoT



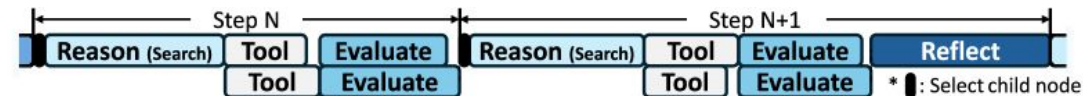
ReAct



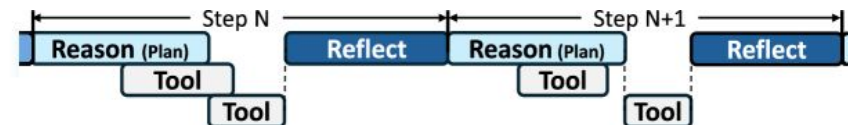
Reflexion



LATS



LLMCompiler



Read it as: each block is one LLM call or one tool call



Methodology

Models and test platform

Resource	8B (default)	70B (size study)
GPU	1 × A100 (40GB)	8 × A100 (40GB each)
vCPUs	12	96
Host memory	85GB	680GB

Llama 3.1 Instruct; prefix caching enabled by default.

vLLM 0.6.6 · PyTorch 2.6 · CUDA 12.8



Methodology

Experiments 1–2: one request, and then many requests

1. One request at a time

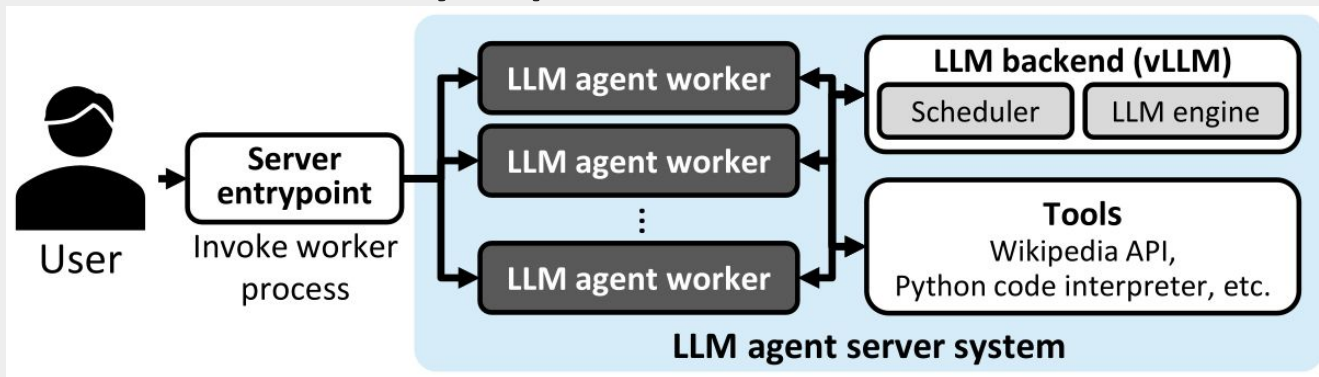
- Every agent on all four tasks
- Count LLM and tool calls

2. Many requests at once

- ReAct on HotpotQA and WebShop
- Baseline: ShareGPT chatbot
- Raise the request rate (QPS)
- Measure P95 latency
- Prefix caching on vs. off

Methodology

The test server for many requests



- **Workers** run requests in parallel; each step calls the LLM or a tool
- **vLLM** batches LLM calls from all workers, first come first served

Key Results I: Serving Overhead

Dynamic Reasoning Increases Per-Request Work

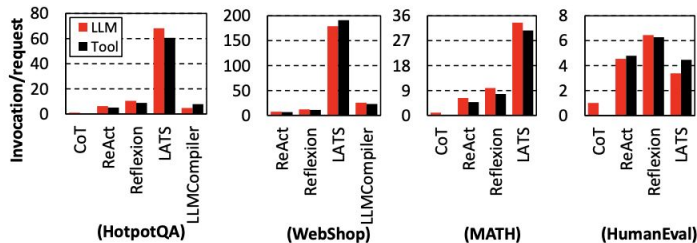


Fig. 4. Average number of LLM and tool invocations per request.

- Tool-augmented agents require many more LLM/tool calls
- LATS is the most extreme case

Agent Serving Has Lower Throughput, but Caching Helps

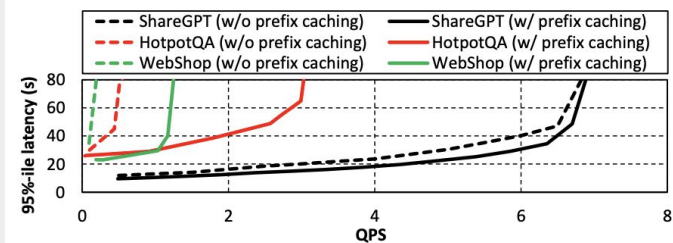


Fig. 11. 95th percentile latency for chatbot (ShareGPT) and ReAct-based AI agents (HotpotQA and WebShop) as QPS rates increase, with (solid line) and without (dashed line) prefix caching enabled.

- Agent workloads saturate at much lower QPS than ShareGPT
- Prefix caching helps because long prefixes are repeatedly reused



Methodology

Experiments 3–4: accuracy, cost, and energy

3. Accuracy vs. cost

- ReAct, Reflexion, LATS, LLMCompiler
- Different step limits and prompt examples
- 50 questions per setting
- Accuracy, latency, accuracy $\div$ latency

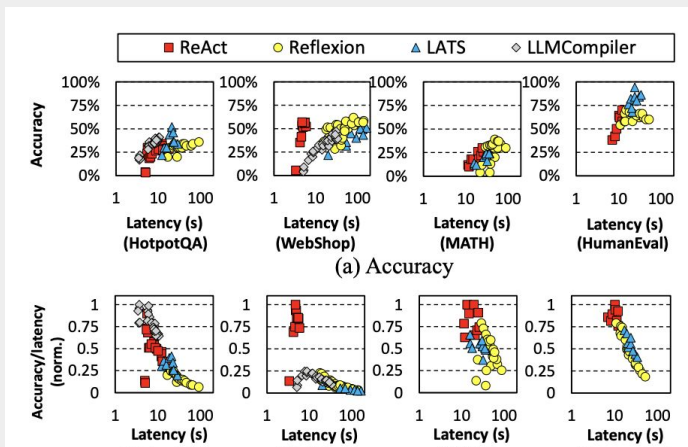
4. GPU energy per query

- Most accurate Reflexion and LATS
- HotpotQA; 8B and 70B models
- Energy per query (Wh)
- Compared with ShareGPT chatbot

Key Results II: Test-Time Scaling Cost

More Reasoning Shows
Diminishing Returns

Agent Test-Time Scaling
Has Large Energy Cost



- Higher accuracy generally requires more latency
- Accuracy per latency decreases as reasoning becomes more expensive

TABLE III
ACCURACY, LATENCY, AND GPU ENERGY CONSUMPTION WHEN SERVICING A SINGLE AGENT REQUEST ON HOTPOTQA. NUMBERS IN PARENTHESES INDICATE THE RELATIVE INCREASE OVER SHAREGPT (THE CONVENTIONAL SINGLE-TURN INFERENCE).

		Accuracy (%)	Latency (seconds)	Energy (Wh/query)
8B	ShareGPT	–	4.23 (1×)	0.32 (1×)
	Reflexion	38	649.34 (153.7×)	41.53 (130.9×)
	LATS	80	380.90 (90.1×)	22.76 (71.7×)
70B	ShareGPT	–	6.40 (1×)	2.55 (1×)
	Reflexion	67	720.00 (112.6×)	348.41 (136.5×)
	LATS	82	305.67 (47.8×)	158.48 (62.1×)

- Agent reasoning raises GPU energy/query by about 62×–136×
- Higher reasoning quality brings substantial infrastructure cost

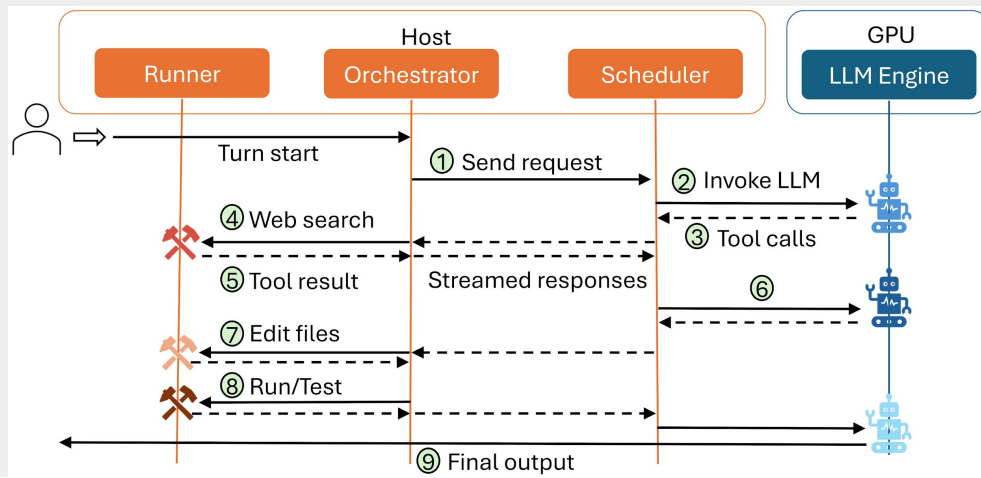
Towards Understanding, Analyzing, and Optimizing Agentic AI Execution: A CPU-Centric Perspective

**Ritik Raj, Souvik Kundu, Ishita Vohra, Hong Wang,
Tushar Krishna**

Background and Motivation

Why Agents Change the Systems Workload

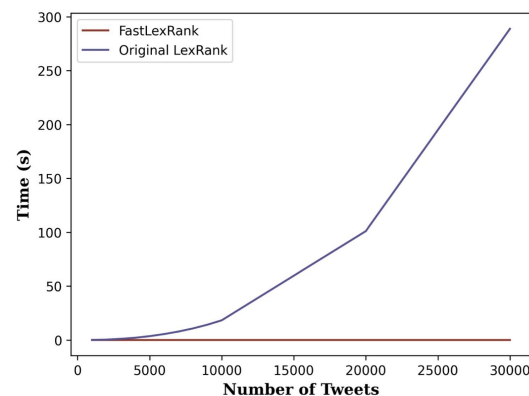
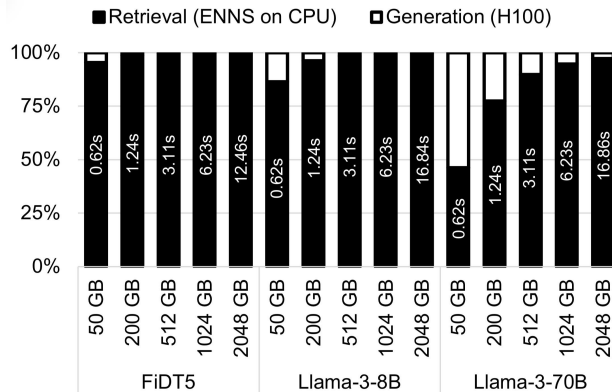
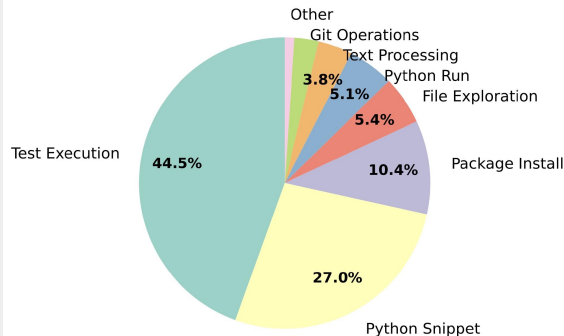
- An agent request is more than one LLM inference.
- LLM inference typically runs on the GPU.
- Tool execution and orchestration often run on the CPU.
- Agents repeatedly alternate between reasoning and tool use.



Agent Tools Create Diverse CPU Workloads

- Representative CPU-side tools:
 - Bash / Python execution → compute + I/O intensive
 - Exact Nearest Neighbor Search (ENNS) → memory + I/O intensive
 - Lexical summarization → compute intensive
- Different tools stress different CPU resources.

(b) Bash Command Time by Category



Agent Serving Requires Joint CPU–GPU Optimization

- **GPU side is well optimized**
 - Orca (OSDI '22)
 - PagedAttention/vLLM (SOSP '23)
 - SGLang (NeurIPS '24)
- **CPU/tool optimization also exists**
 - IKS (ASPLOS '25)
 - Conveyor (arXiv, 2024)

**The missing piece: characterize where the bottleneck is,
then jointly optimize CPU and GPU utilization.**

Methodology

Compile-time Characterization

Agentic Workload	Compile-time Characterization			Tools	Application
	Orchestrator	Path	Flow		
Toolformer [59]	LLM	Dynamic	Single-step	<u>Calculator API</u> , <u>Calendar</u>	<u>MLQA</u> , <u>Math</u>
SWE-Agent [75]	LLM	Static	Multi-step	<u>Bash (File I/O and Python Execution)</u>	<u>SDE</u> , <u>Data analysis</u>
RAG (Haystack [14])	Python code (Host)	Static	Single-step	<u>Web search</u> , <u>Document Retrieval</u>	<u>RAG QA</u>
ChemCrow [10]	LLM	Dynamic	Multi-step	<u>Conformer Gen tool</u> , <u>Reaction Tools</u>	<u>Chemistry Research</u>
Web-Augmented Agent (LangChain[43])	Python code (Host)	Static	Single-step	<u>Web search</u> , <u>summarizer</u>	<u>Web-based QA</u> , <u>DevOps</u>

- **Orchestrator:** who drives the workflow, the LLM or host Python code
- **Path:** fixed in advance (static) or decided at run time (dynamic)
- **Flow:** one pass (single-step) or repeated act–observe loops (multi-step)
- **Models:** small LMs keep the loops fast; tools handle computation and retrieval



Profiling

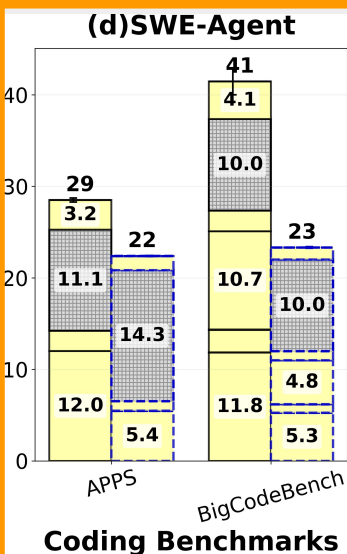
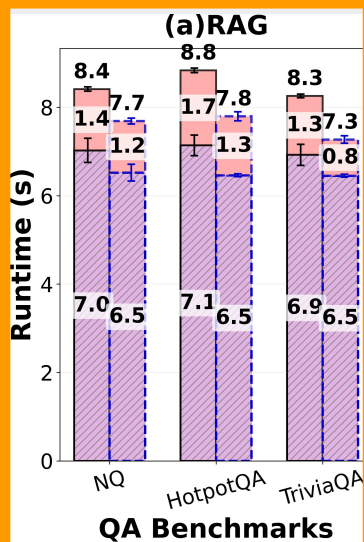
Three questions, three takeaways

Where does the time go?	CPU tools can take most of the latency	Speeding up only the GPU is not enough
How do the CPU and GPU interact?	A faster GPU shifts the bottleneck to the CPU	Optimize the CPU and GPU together
How does throughput scale?	CPU tools hit their limit before the GPU	Limit CPU concurrency and overlap CPU-GPU work



Profiling

Takeaways 1–2: CPU tools can dominate latency



LLM Inference (GPU)

■ GPT-OSS-20B (vLLM)

■ Qwen2.5-Coder-32B (vLLM)

Tools (CPU)

■ ENNS Retrieval

■ Bash/Python Execution

Hardware

□ System 1

□ System 2

(a) RAG: retrieval on the CPU takes over 80% of the time

(d) SWE-Agent: tools reach 65% on Sys 2 (faster GPU)

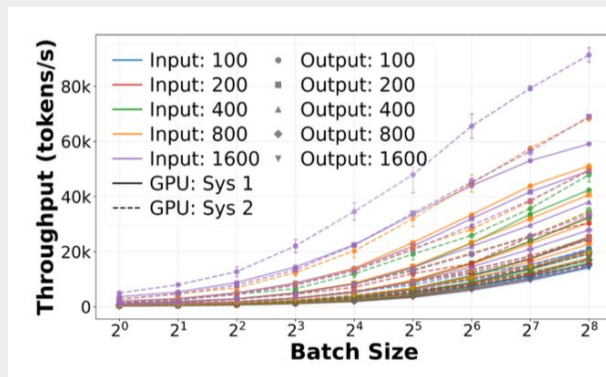
→ **Speeding up only the GPU is not enough**

Sys 1: 64-core Intel CPU + RTX Pro 6000 GPU

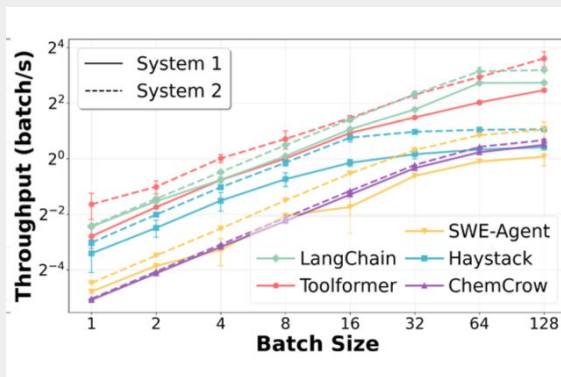
Sys 2: 72-core Grace CPU + H200 GPU (faster GPU)

Profiling

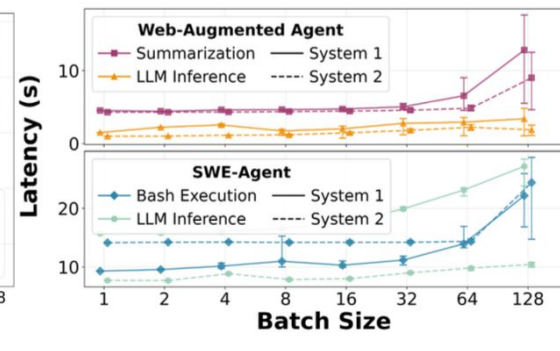
Takeaway 3: CPU tools hit their limit first → COMB and MAS



GPU only: LLM inference (tokens/s)



Full agent workflow: requests/s and stage latency



Proposed Optimizations

COMB (CPU-Aware Overlapped Micro-Batching): problem → idea

From profiling: CPU tool work and GPU inference run in separate phases

The CPU gets overloaded

Too many requests at once: CPU tools compete and run $\sim 2\times$ slower (batch 128)



Cap the batch size

Split a large batch into smaller groups, so the CPU is not overloaded

The CPU and GPU take turns

While tools run on the CPU, the GPU waits, and the other way around



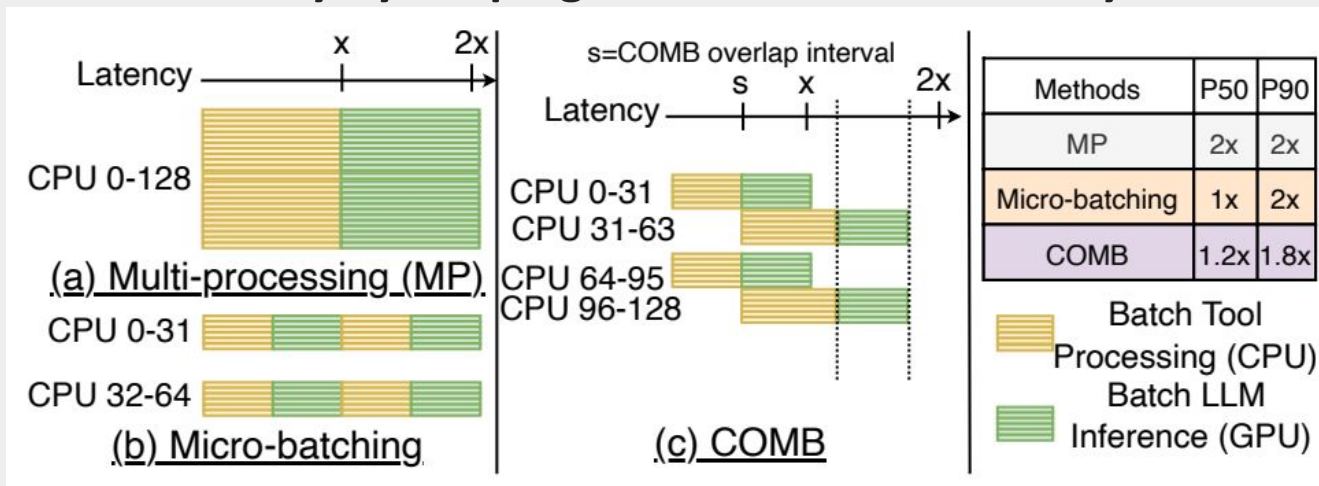
Overlap the groups

While one group uses the GPU, the next group runs its tools on the CPU

In live serving: COMB limits how many CPU-heavy requests are active (N_{cap})

Proposed Optimizations

COMB cuts latency by keeping both CPU and GPU busy



Times are illustrative



Proposed Optimizations

MAS (Mixed Agentic Scheduling): problem → idea

Two request types: CPU-heavy (call tools) and GPU-heavy (LLM only)

One queue is unfair

With a single queue, the more common type takes most slots; the other waits



A separate limit per type

Each type gets its own slots, so both the CPU and GPU stay busy

Demand changes over time

When one type surges, its own slots alone are not enough



A small shared reserve

Extra requests of either type can borrow from a shared pool

Link to COMB: the CPU-heavy limit is COMB's N_{cap}

Proposed Optimizations

MAS: each type keeps its own slots, so neither is crowded out

224 active slots in total



Evaluation

Evaluation

Setup: four experiments

COMB: one batch	128 requests at once: all vs. 2×64 vs. COMB
COMB: live serving	Requests keep arriving (Sys 2): limit 256 vs. 48–96
MAS: mixed requests	FCFS vs. MAS with 224 slots; 25/50/75% GPU-heavy

Main metrics: P50 (median) and P90 (tail) latency

COMB Improves Latency and Throughput Together

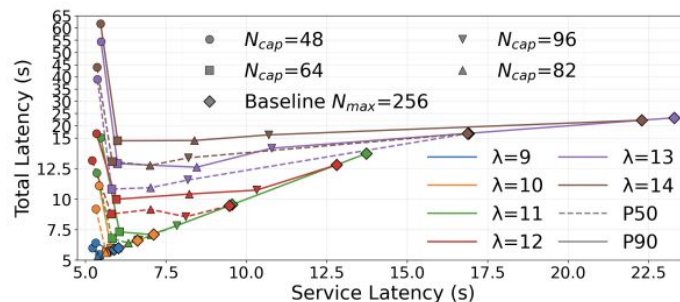
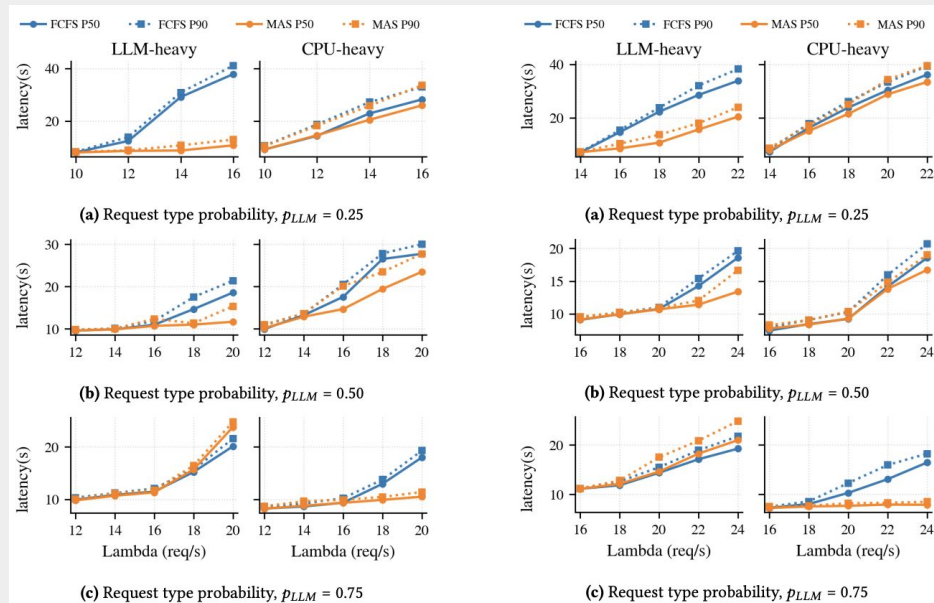


Figure 6. In open-loop serving system with $\lambda = 9 - 15$ req/s arrival-rate for Web-Augmented Agent on Sys 2.

- Total latency improves by 1.6x/1.8x, and throughput improves by 1.7x
- $N_{cap}=64$ keeps CPU utilization balanced under high load
- At $\lambda=13$ req/s, COMB reduces service latency by 2.9x/3.9x at P50/P90

MAS Protects Minority Workloads in Mixed Traffic

- With one FCFS queue, the majority request type can consume admission slots even if the minority stresses a different resource
- MAS gives CPU-heavy and GPU-heavy requests separate capacity plus a small shared reserve
- It improves minority GPU-heavy latency by up to $2.37\times$ / $2.49\times$, and minority CPU-heavy latency by up to $2.09\times$ / $2.15\times$



Paper Fig. 7/8: MAS vs. FCFS under bursty mixed arrivals on Sys 1 and Sys 2

Discussions



Main Takeaways

- CPU tools can limit end-to-end performance even when GPU inference becomes faster
- COMB improves CPU-heavy serving by capping concurrency and overlapping CPU/GPU stages
- MAS protects minority request types through separate admission limits

Limitations

- Overlap gains depend on CPU capacity and tuned caps. On the 16-core platform, micro-batching alone can be better
- Scheduling tests cover selected workloads and systems. Web-Agent uses cached HTML, excluding live network variability
- Energy profiling identifies CPU costs, but does not establish COMB/MAS energy savings



Agents as a New System Workload

Task quality and serving cost should be evaluated together

CPU-centric execution

- CPU tools can bottleneck serving
- COMB/MAS improve CPU-GPU coordination

Dynamic reasoning

- Repeated calls raise serving cost
- Caching helps. Reasoning gains diminish

Workflow design sets demand. Scheduling shapes efficiency.

Thank You!

Dynamic Reasoning Increases Per-Request Work

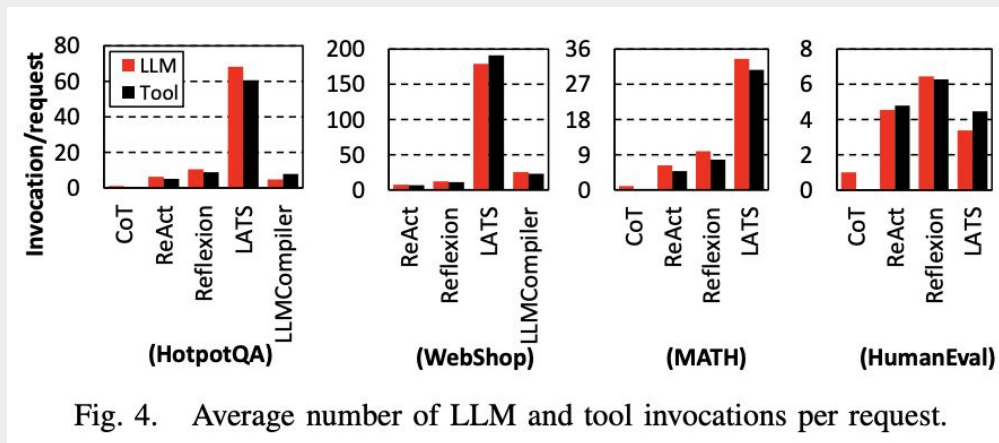


Fig. 4. Average number of LLM and tool invocations per request.

- One user request can trigger many LLM and tool invocations
- Latency comes from both LLM inference and tool execution
- The sequential dependency makes a single agent request hard to parallelize internally for dependency-bound reasoning/tool stages

Agent Serving Has Lower Throughput, but Caching Helps

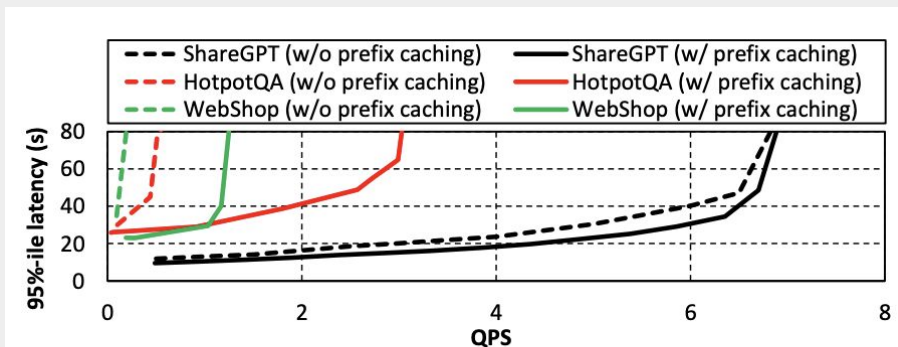


Fig. 11. 95th percentile latency for chatbot (ShareGPT) and ReAct-based AI agents (HotpotQA and WebShop) as QPS rates increase, with (solid line) and without (dashed line) prefix caching enabled.

- ReAct reaches lower sustainable QPS as each request contains repeated LLM/tool stages
- Prefix caching helps agents because many LLM calls reuse long shared prefixes
- Repeated contexts create overhead, but also make prefix caching especially valuable

More Reasoning Shows Diminishing Returns

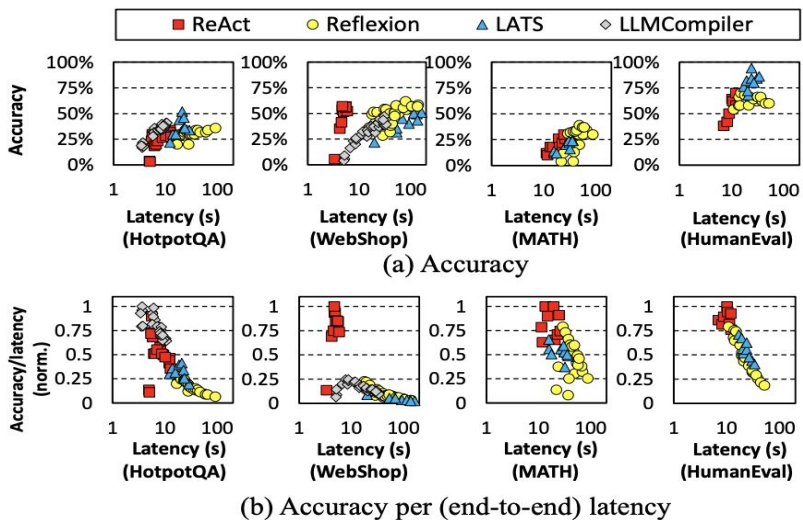


Fig. 13. Accuracy and cost-efficiency of AI agent design points. (a) Accuracy vs. latency and (b) Accuracy per latency, illustrating how efficiently each configuration translates cost into task performance.

- Higher accuracy usually requires more latency, but cost-efficiency falls after the best trade-off point
- Sequential scaling improves accuracy slowly, while parallel scaling trades memory for lower latency
- Test-time scaling should be budgeted by efficiency, not only by maximum accuracy

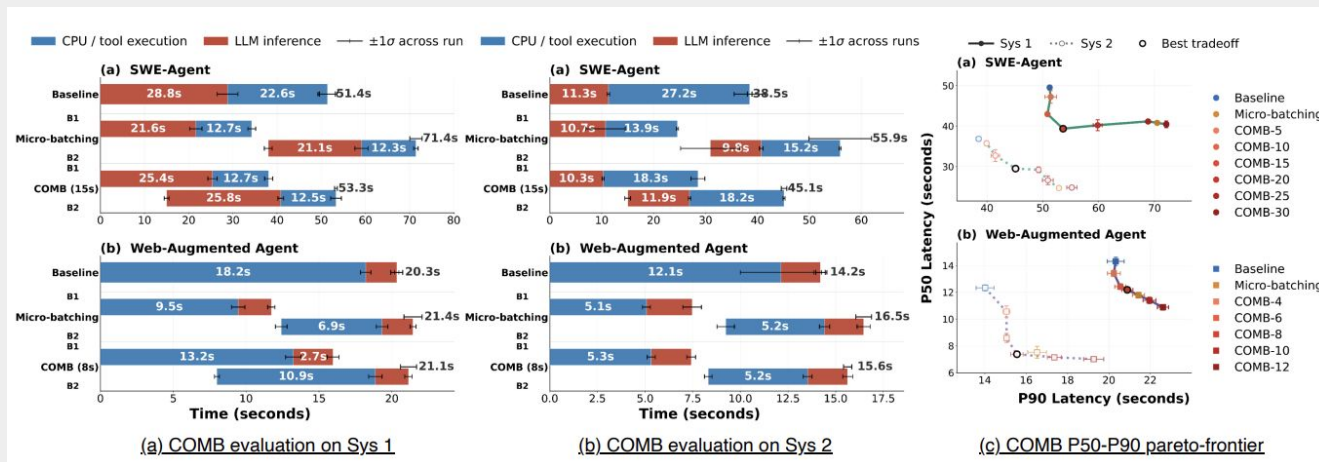
Agent Test-Time Scaling Has Large Energy Cost

TABLE III
ACCURACY, LATENCY, AND GPU ENERGY CONSUMPTION WHEN
SERVICING A SINGLE AGENT REQUEST ON HOTPOTQA. NUMBERS IN
PARENTHESES INDICATE THE RELATIVE INCREASE OVER SHAREGPT (THE
CONVENTIONAL SINGLE-TURN INFERENCE).

		Accuracy (%)	Latency (seconds)	Energy (Wh/query)
8B	ShareGPT	–	4.23 (1×)	0.32 (1×)
	Reflexion	38	649.34 (153.7×)	41.53 (130.9×)
	LATS	80	380.90 (90.1×)	22.76 (71.7×)
70B	ShareGPT	–	6.40 (1×)	2.55 (1×)
	Reflexion	67	720.00 (112.6×)	348.41 (136.5×)
	LATS	82	305.67 (47.8×)	158.48 (62.1×)

- Agent-based reasoning can raise per-query GPU energy by 62.1x-136.5x
- More reasoning is a resource allocation problem, not a free path to higher quality

Experiment 1: COMB on One Fixed Batch



- COMB caps the CPU-heavy micro-batch at 64 and overlaps CPU tool execution with GPU inference
- Web-Augmented Agent benefits most when throughput has saturated
- For SWE-Agent, plain micro-batching can hurt tail latency; overlap creates a better P50/P90 trade-off

Experiment 2: COMB in Live Open-Loop Serving

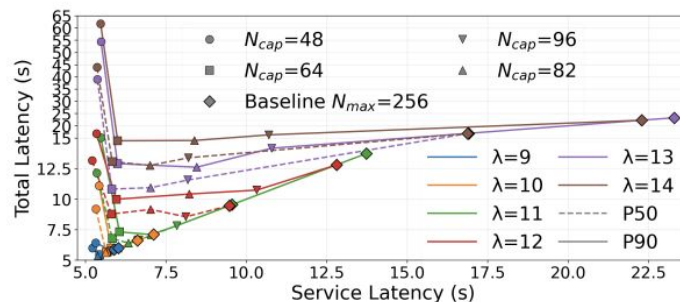
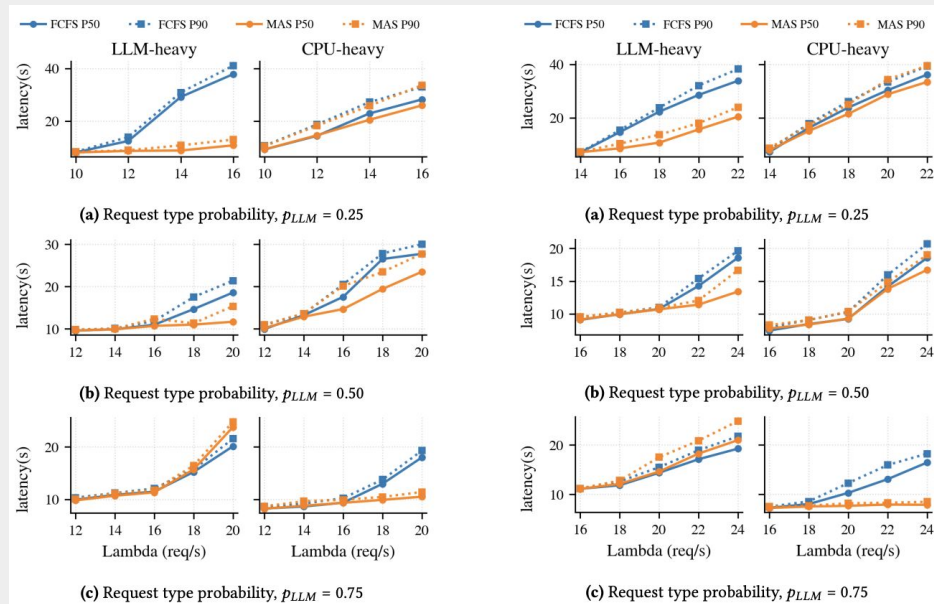


Figure 6. In open-loop serving system with $\lambda = 9 - 15$ req/s arrival-rate for Web-Augmented Agent on Sys 2.

- Total latency improves by 1.6x/1.8x, and throughput improves by 1.7x
- $N_{cap}=64$ keeps CPU utilization balanced under high load
- At $\lambda=13$ req/s, COMB reduces service latency by 2.9x/3.9x at P50/P90

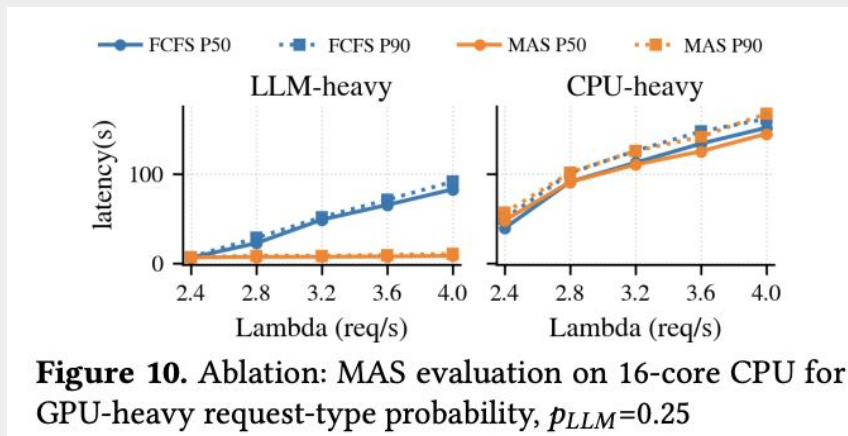
Experiment 3: MAS for Mixed Agentic Requests

- MAS uses iso-concurrency with 224 slots: 64 CPU-heavy, 128 GPU-heavy, and 32 shared
- When $p_{LLM}=0.25$, MAS protects GPU-heavy minority requests: up to 2.37x/2.49x P50/P90 gain on Sys1
- When $p_{LLM}=0.75$, MAS protects CPU-heavy minority requests: 2.09x/2.15x P50/P90 gain on Sys 2



Paper Fig. 7/8: MAS vs. FCFS under bursty mixed arrivals on Sys 1 and Sys 2

Ablation 1: CPU Constrained System



- For COMB, micro-batching helps, but aggressive overlap can reintroduce CPU contention
- For MAS, separating CPU-heavy and GPU-heavy admission becomes even more important
- With $p_{LLM}=0.25$, MAS improves GPU-heavy minority latency by up to $10.1\times$ / $8.8\times$ at P50 / P90



Ablation 2: Energy Usage

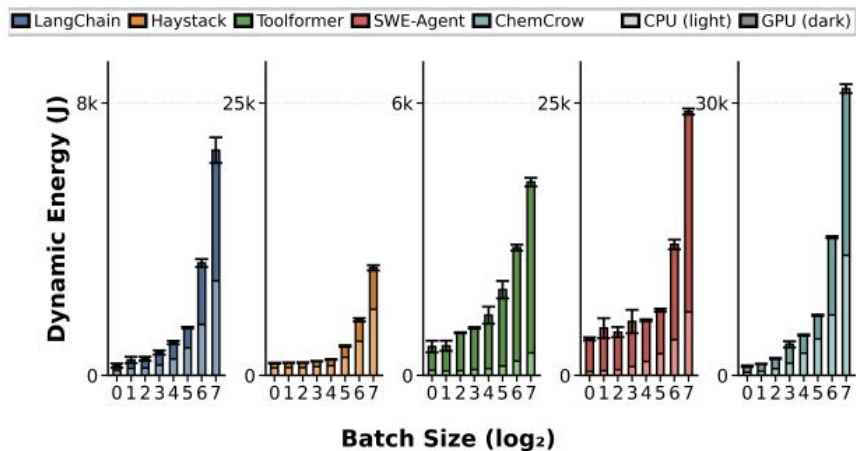


Figure 11. CPU and GPU dynamic energy consumption.

- RAG shows a uniform CPU dynamic energy contribution of about 61%
- Web-Augmented Agent reaches 57% CPU dynamic energy at batch size 32
- ChemCrow peaks around 60% CPU dynamic energy; SWE-Agent rises to 33% at batch size 128
- Toolformer remains mostly GPU-heavy, showing that energy bottlenecks also depend on the tool mix